3.2 指令系统

设计CPU的一般过程：

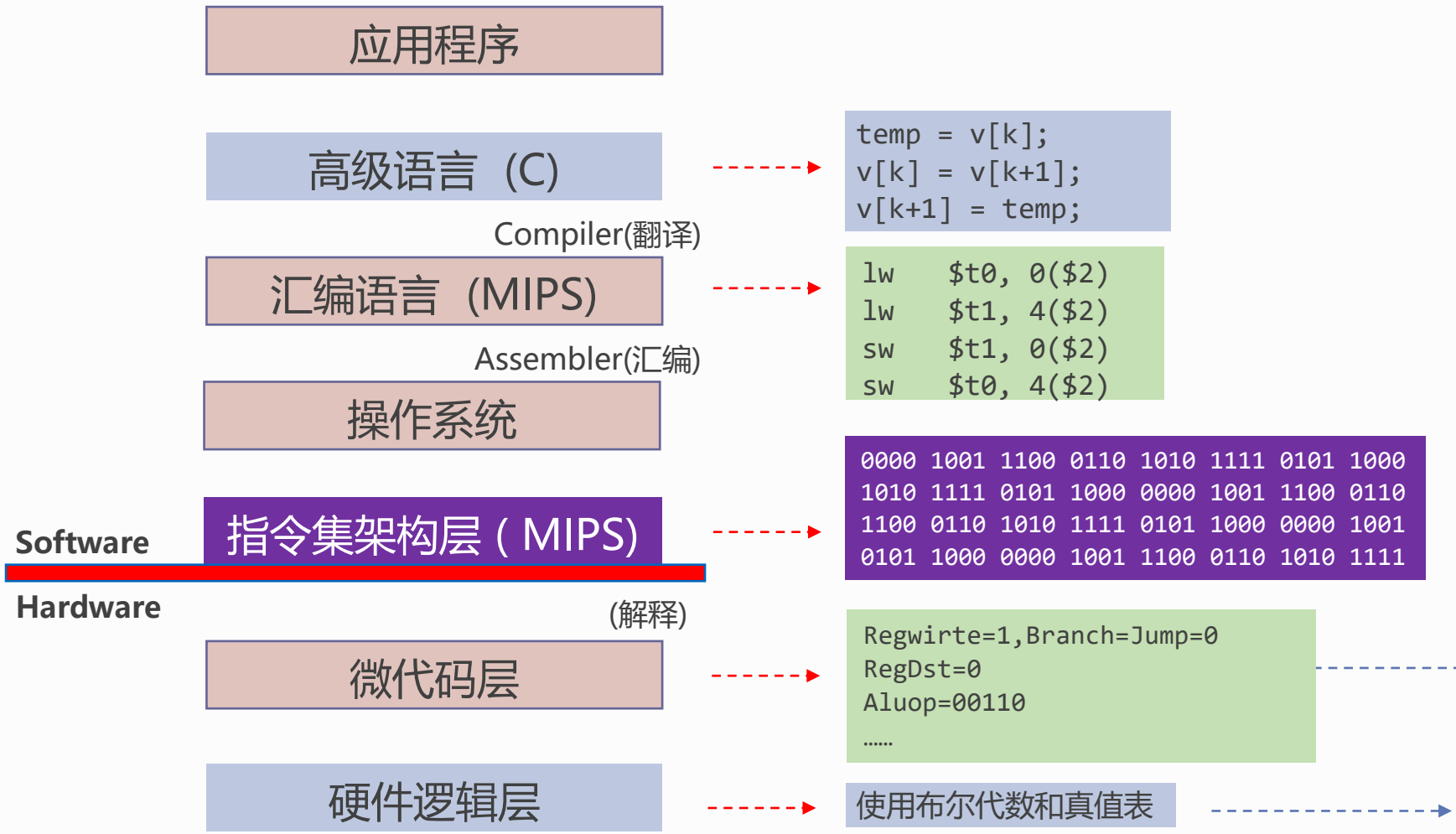
指令系统→ 数据通路→ 控制器→ CPU定型

本节主要分析：√指令格式

√指令涉及的寻址方式

√指令功能和类型

1 指令系统认识

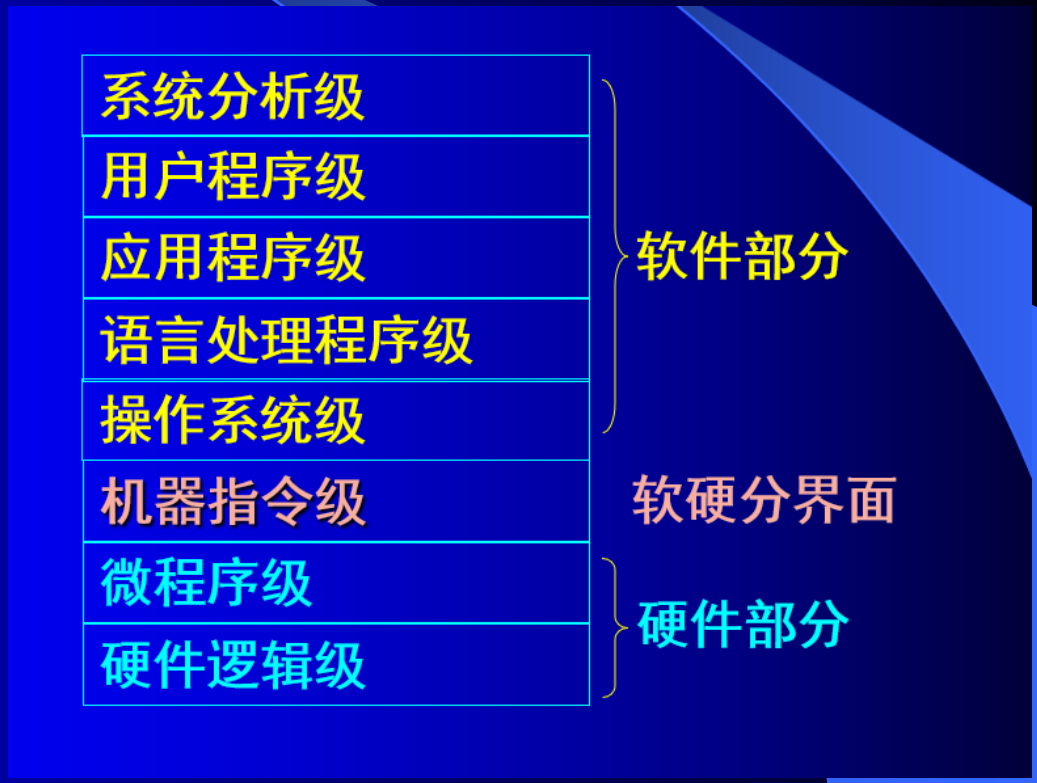


- 是计算机能直接识别、执行的某种操作命令（机器指令）
- 冯诺依曼机构计算机“程序控制”的载体。
- 硬件设计的依据、软件设计的基础
- 硬件和软件间的界面、程序员操作计算机的接口
- 直接影响计算机系统性能和基本功能

⊙指令：*instruction*，计算机执行某类操作的信息的集合，是CPU工作的主要依据。

⊙指令集：*instruction set*，处理器能执行的全体指令的集合（CISC、RISC）

- √ *决定了计算机的硬件功能*
- √ *计算机中软硬件的分界面*



⊙指令：*instruction*，计算机执行某类操作的信息的集合，是CPU工作的主要依据。

⊙指令集：*instruction set*，处理器能执行的全体指令的集合（CISC、RISC）

- √ *决定了计算机的硬件功能*
- √ *计算机中软硬件的分界面*

⊙ 指令/数据字：指令/数据的一组二进制代码。

⊙ 指令/数据字长：指令/数据二进制代码位数

⊙ 机器字长：计算机能够直接处理的二进制数据的位数 = 寄存器的宽度。

3.2.1 指令集类型

1、CISC（Complex Instruction Set Computing）

复杂指令集计算

早期计算机部件昂贵、速度慢，为了扩展硬件功能，不得不将更多更复杂指令加入到指令系统，以提高计算机的处理能力 → 复杂指令集

2、RISC（Reduced Instruction Set Computing）

精简指令集计算

随着半导体技术进步，80年代开始逐渐直接通过硬件方式，而不是扩充指令来实现复杂功能，指令规模逐渐缩小、指令进一步简化 → 精简指令集

※CISC→RISC的技术背景

①2—8规律

CISC中的不同指令使用频率悬殊：

简单指令(约占20%的)约占80%的使用频率，复杂指令(约占80%)只占大约20%的使用频率。

②不利于VLSI工艺

为实现大量的复杂指令，控制逻辑极不规整，给VLSI工艺造成很大困难。

③ 主存技术的发展

一般通过保存在控制存储器中的微程序来实现复杂指令，70年代后期开始用DRAM做主存，使主存与控制存储器的速度相当，因此很多复杂指令不必再用微程序来实现，可用简单指令构成的子程序实现等效功能。

→CISC的特点：

①指令数量多；

②指令长度可以不固定，指令格式和寻址方式多样；

③很多指令会涉及存储器读写操作，指令周期长；

④一般在通用处理器中使用；

→RISC的特点：

①指令数量少；

②指令长度固定，指令格式和寻址方式种类也少；

③一般只有少量指令(如取数/存数) 才会读写存储器，其余指令只涉及CPU内部寄存器，指令周期短；

④一般在高端服务器中使用；

3.2.2 指令格式

一条指令应该包含以下要素：

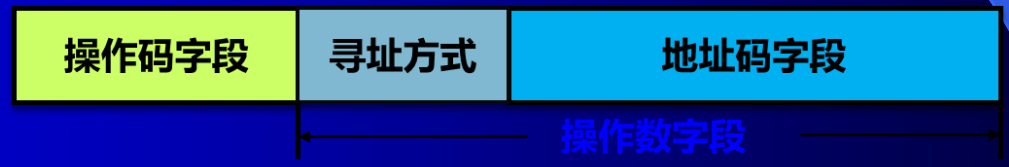
- 指令要求计算机处理什么数据？
- 指令要求计算机对数据做什么处理？
- 计算机怎样才能得到要处理的数据？

指令的操作数需要解决的问题

指令的操作码需要解决的问题

指令的寻址方式需要解决的问题

指令的基本格式



操作码OP	地址码A /操作数D

1个　　1个或几个（广义）

3.2.2 指令格式

一条指令应该包含以下要素：

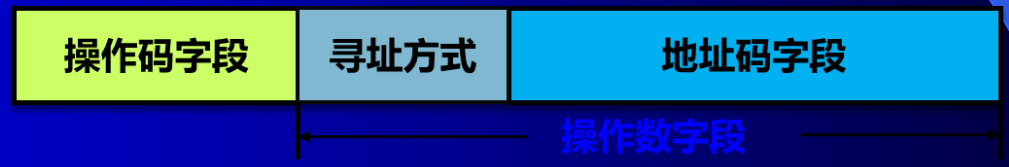
- 指令要求计算机处理什么数据？
- 指令要求计算机对数据做什么处理？
- 计算机怎样才能得到要处理的数据？

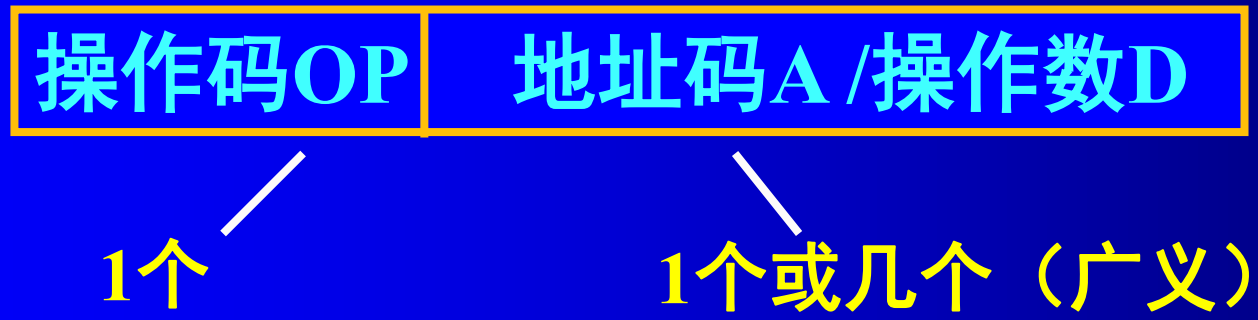
指令的操作数需要解决的问题

指令的操作码需要解决的问题

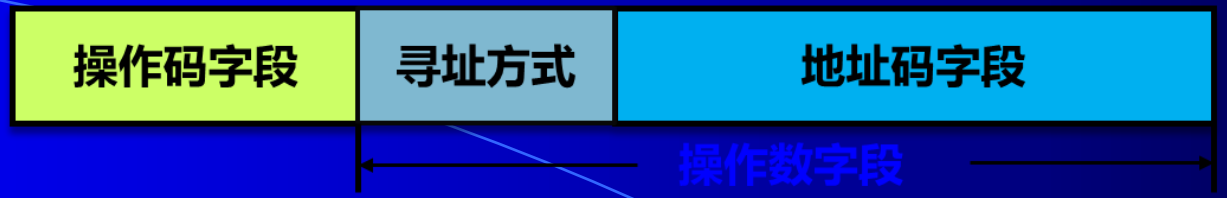
指令的寻址方式需要解决的问题

指令的基本格式





说明：



操作码OP	地址码A /操作数D

问题：对于一条指令，如何知道当前指令采用的是何种寻址方式？？

两种典型途径：

（1）通过指令的操作码字段来隐含说明；

（2）在指令中分别设置专用的寻址方式附加说明字段。

1. 指令字长

- 定长指令格式：规整、便于控制，早期一般采用，16字节
- 变长指令格式：合理利用存储空间、提高取指令的效率，如超长指令集

操作码OP	地址码A /操作数D

注：指令字长是字节（8位）的整数倍，比如单字节、双字节、三字节等。

指令寄存器的位数取决于指令字长

2.指令的 地址结构

操作码OP	地址码A/操作数D

指令中提供的地址 < 地址偏移量/立即数
寄存器编号

（1）指令中提供地址的方式

√ 显式地址方式

→指令中明确指明地址码（直接/间接给出）

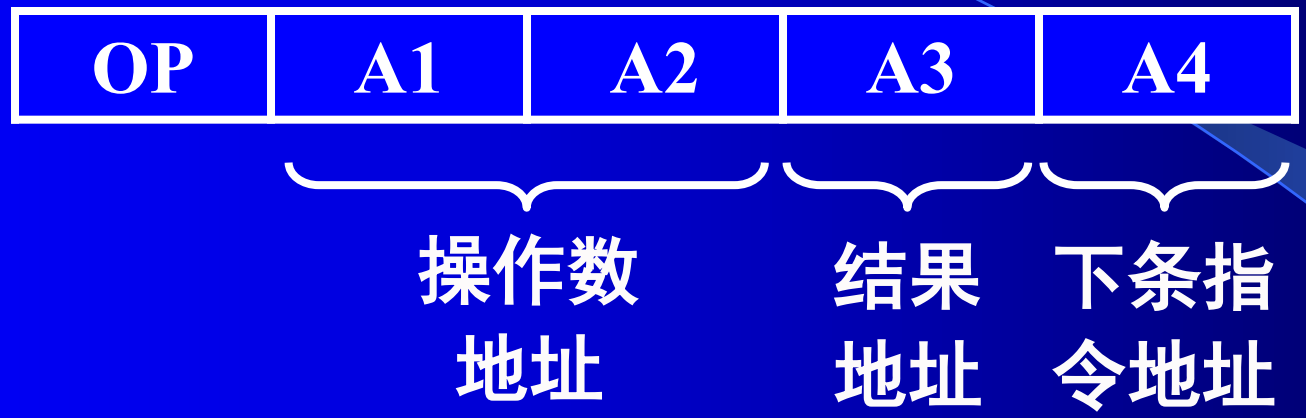
√ 隐式地址方式

→地址码隐含约定，不在指令中出现。

使用隐式地址，可以减少指令中的地址数量，从而简化地址结构。

（2） 常见的地址结构类型

✓ 四地址结构指令



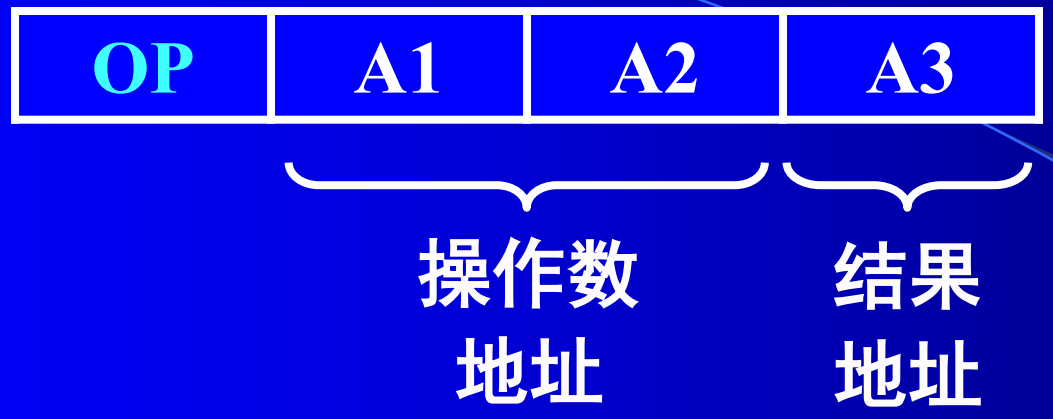
功能：(A1)OP(A2)→A3

A4→PC

一般用PC寄存器指示下条指令的地址。

四地址结构指令在RISC中很少会使用。

✓三地址结构指令



需要深刻理解 “+1”

存储1条指令占用的字节单元数

与指令字长有关!

指令字长16位，PC+“1”实际上为PC+2

功能：

(A1) OP (A2) → A3

(PC) + → PC　　如：ADD rd, rs, rt

↓

自动修改PC的内容，使PC指向下一条指令

PC开始时存放程序的首地址，每执行一条指令，PC 加”1”，指出下条指令的地址， 直到程序结束。

1、对于8位字长指令而言，PC顺序寻址时应该增加的常量是(　　)

A. 4　　B. 8　　C. 1　　D. 2

2、对于64位字长指令而言，PC顺序寻址时应该增加的常量是(　　)

A. 4　　B. 8　　C. 1　　D. 2

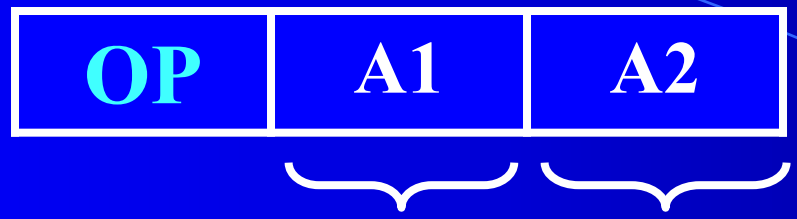
1、字节编址；对于8字长指令而言，PC顺序寻址时应该增加的常量是（ C　）

A. 4　　B. 8　　C. 1　　D. 2

2、字节编址；对于64位字长指令而言，PC顺序寻址时应该增加的常量是（ B　）

A. 4　　B. 8　　C. 1　　D. 2

✓二地址结构指令



功能：

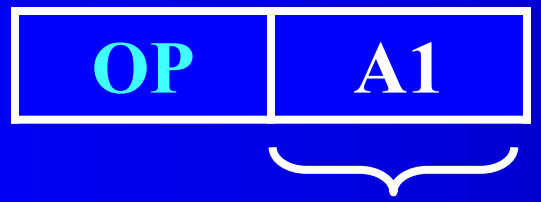
(A1) OP (A2) → A1

(PC)+ → PC　　如：ADD R1, R0

↓

自动修改PC的内容，使PC指向下一条指令

✓ 一地址结构指令



源/目的地址　　隐含约定

双操作数：A1 U $[PC]_H$→PC　如：J addr

addr:指针传递

单操作数：

OP(A1) → A1

(PC)+ → PC　如：INC R0

inc：+1

自动修改PC的内容，使PC指向下一条指令

求反、加1、减1、求补等

✓ 零地址结构指令



如停机指令、堆栈类指令等

功能：

①用于处理机的特殊控制（如HLT（暂停），NOP（空操作））。

②针对隐含约定的寄存器，如返回指令：

RST

↓

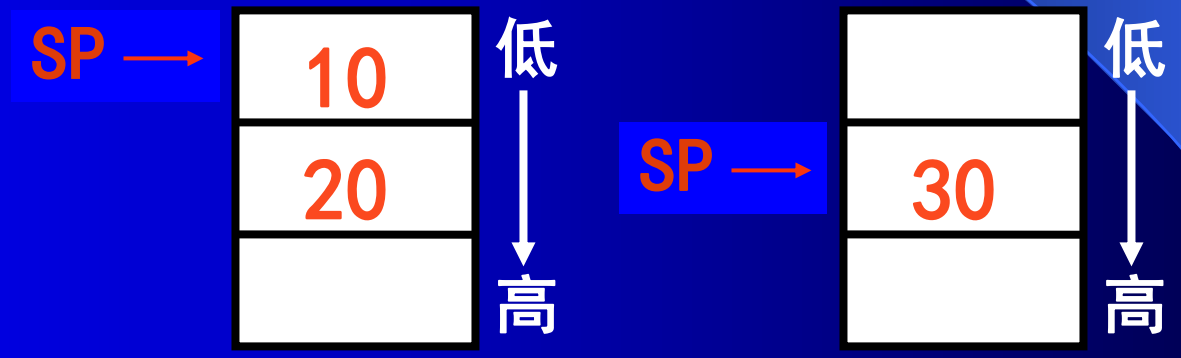
隐含操作：(SP)→PC；SP^{+} →SP；

把堆栈栈顶单元保存的返回地址打入PC。

③用于堆栈：((SP))OP((SP)+1)⟶(SP)+1

(SP)+1⟶ SP

例. ADD；　执行前：　　执行后：



向上生长方式堆栈 入栈：(sp)-1后压入　出栈：先弹出后(sp)+1

向下生长堆栈 入栈：(sp)+1后压入 出栈：先弹出后(sp)-1

栈顶固定堆栈：栈底浮动，栈顶固定

3. 操作码结构

(1) 定长操作码

各指令OP的位置、位数固定相同。

(2) 扩展操作码

各指令OP的位置、位数不固定，根据需要变化(设置扩展标志)。

操作码OP	地址码A /操作数D

(1) 定长操作码

各指令OP的位置、位数固定相同。

1 操作码的位数决定了操作类型的多少，即操作码字段长度决定指令系统规模

2 每条指令对应一个操作码

例如：若操作码字段占6位，则采用定长操作码时指令系统最多有$2^6=64$条指令。

定长操作码特点：简化硬件设计，减少指令译码时间，广泛用于字长较长的大、 中型计算机和超级小型计算机中。

(2) 扩展操作码

各指令OP的位置、位数不固定，根据需要变化

（关键在于设置扩展标志）。

◆扩展操作码的方法：

在指令字中用一个固定长度的字段表示基本操作码，而对于一部分不需要某个地址码的指令，可将操作码扩充到地址码字段。

例如：假设一台计算机指令字长16位，其中4位为基本操作码字段，另外3个4位长的地址码字段位A1、A2、A3。请采用“扩展操作码技术”设计61条指令，其中三地址指令15条，二地址指令15条，一地址指令15条，零地址指令16条。

4 位操作码

0000 A1 A2 A3

……

1110 A1 A2 A3

15条三地址指令

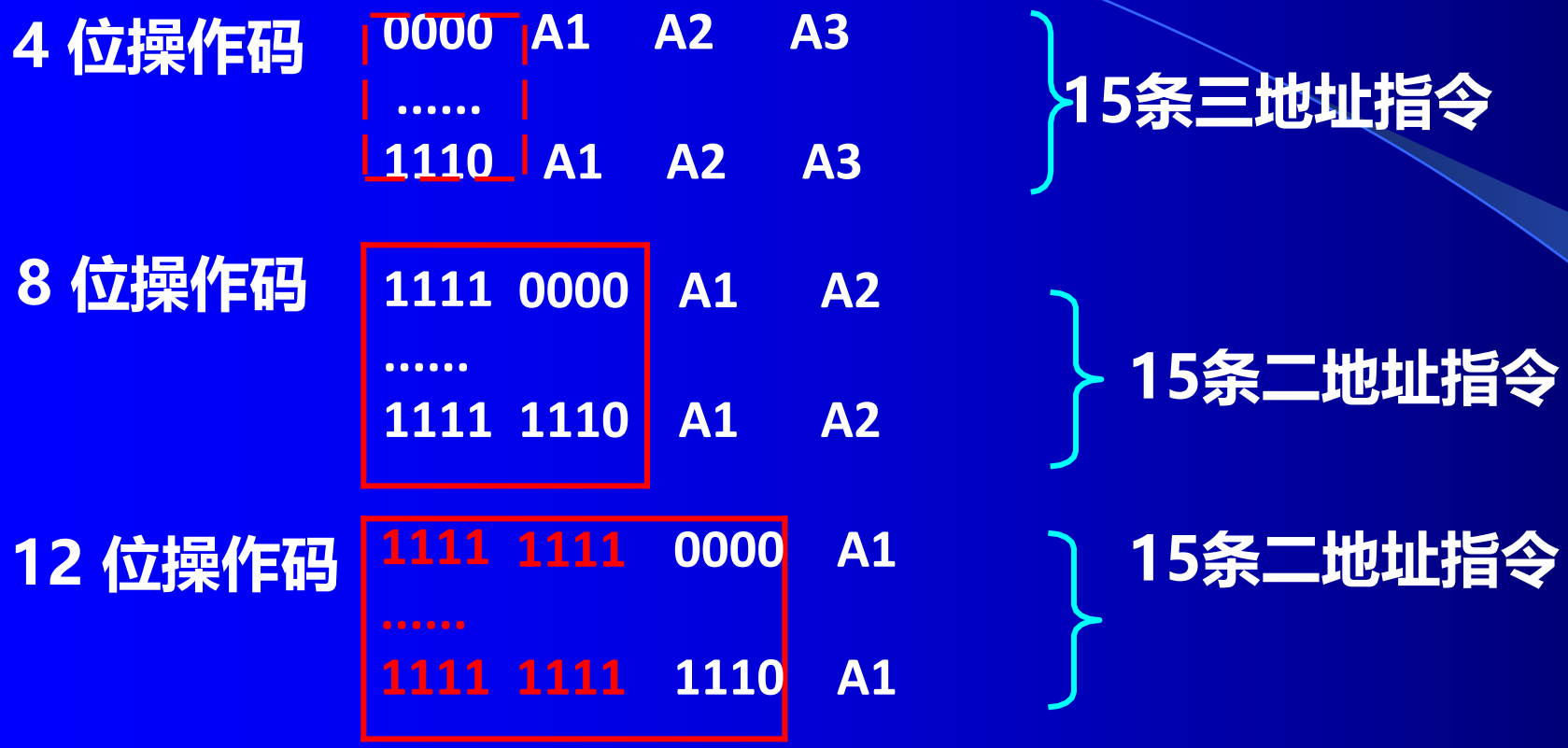
8 位操作码

1111 0000 A1 A2

……

1111 1110 A1 A2

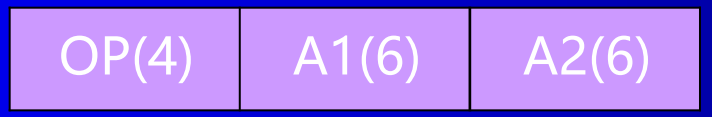
15条二地址指令


4 位操作码
0000 A1 A2 A3
……
1110 A1 A2 A3
15条三地址指令
8 位操作码
1111 0000 A1 A2
……
1111 1110 A1 A2
15条二地址指令
12 位操作码
1111 1111 0000 A1
……
1111 1111 1110 A1
15条二地址指令


4 位操作码
0000 A1 A2 A3
.....
1110 A1 A2 A3
15条三地址指令
8 位操作码
1111 0000 A1 A2
.....
1111 1110 A1 A2
15条二地址指令
12 位操作码
1111 1111 0000 A1
.....
1111 1111 1110 A1
15条一地址指令
16 位操作码
1111 1111 1111 0000
.....
1111 1111 1111 1111
16条零地址指令

- 练习： 设某指令系统指令字长16位，每个地址码为6位。若要求设计二地址指令15条、一地址指令34条，问最多还可设计多少条零地址指令？

练习 设某指令系统指令字长16位，每个地址码为6位。若要求设计二地址指令15 条、一地址指令34条，问最多还可设计多少条零地址指令?



解: 操作码按从短码到长码进行扩展编码

假定二地址指令编码: (0000 – 1110)　　共15条，1111作为扩展标识;

则一地址指令编码的全集可表示为: 1111 000000 – 111111(假定扩向A1);　因一地址指令只需34条，可将全集中多余的30种编码作为向零地址指令扩展的标识，假定为: 100010 – 111111

故最多可支持的故零地址指令数为:　30×2^6 条

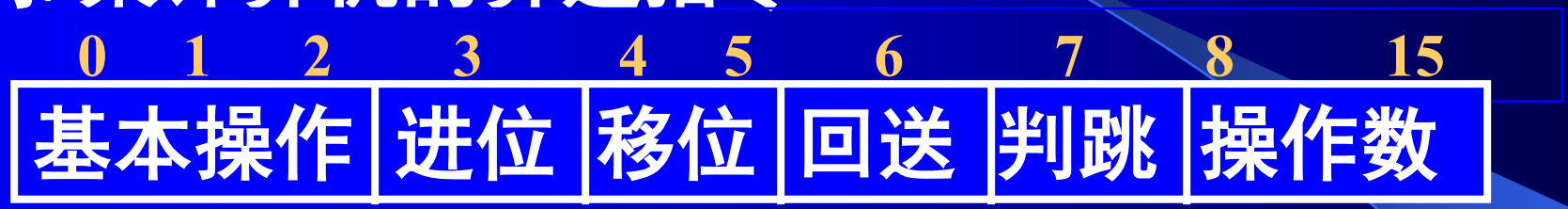
说明：

不管采用什么扩展方式，在设计变长操作码的指令系统时，应尽量让使用频度高的指令占用短的操作码，对使用频度低的指令可占用较长的操作码，这样可以缩短经常使用的指令的译码时间。

(3） 操作码采用方式码编码

操作码分为几部分，每部分表示一种操作。

例. 某计算机的算逻指令



至多：12种操作

4. 指令可能会涉及到的操作数类型



① 地址码数据

寄存器编号或者存储器地址，无符号整数。

② 数值型数据

定点数、浮点数等，一般用补码表示。

③ 字符型数据

通常表示为ASCII码/汉字内码格式。

④ 逻辑型数据

常规二进制代码，不具有数值含义。

5. MIPS32架构的指令格式

- RISC
- 指令字长为：32位
- 寄存器数量：32个

指令类型	指令长度（32位定长）					
	31 ~ 26	25~21	20~16	25~11	10 ~ 6	5 ~ 0
R型	op(6)	rs(5)	rt(5)	rd(5)	shamt	func
I型	op(6)	rs(5)	rt(5)	address/imm (16)		
J型	op(6)	address (26)				

5. MIPS32架构的指令格式

- R型：寄存器-寄存器指令
- I型：寄存器-立即数指令
- J型：分支跳转指令

指令类型	指令长度（32位定长）					
	31 ~ 26	25~21	20~16	25~11	10 ~ 6	5 ~ 0
R型	op(6)	rs(5)	rt(5)	rd(5)	shamt	func
I型	op(6)	rs(5)	rt(5)	address/imm (16)		
J型	op(6)	address (26)				

练习

1、某计算机指令的操作码有6个二进位，这种计算机的指令系统中不同的指令条数至多为（　）　　B

A.8　　B.64　　C.128　　D.256

2、指令寄存器的位数取决于（ ）　　B

A．　存储器的容量　　B．指令字长　　C．机器字长　　D．CPU管脚数

B

3 扩展操作码是（）

A 用来进行指令分类的代码　B 一种指令优化技术，让操作码长度随地址数变化而变化

C指令格式中不同字段设置的操作码　D 操作码字段外用来代表操作类型的代码

练习

4 下列关于指令格式的描述中，正确的是（）　　A B C D

A 对采用定长操作的计算机而言，若需要支持65条指令，则其操作码字段最少需要7位

B 若指令中每个地址字段为4位，则对RR型指令而言，可以使用16个寄存器

C 指令字长确定的情况下，指令的地址字段越多，则其操作码位数就越少

D 计算机硬件是影响指令格式设计的因素之一。

5 假设指令字长16位，操作数的地址码6位，指令有二地址、一地址和零地址三种格式，采用 扩展操作码技术，若二地址指令有8种，零地址指令64种，则一地址指令最多有多少（）

C

A 255 B 342 C 511 D 512

3.2.3 寻址方式

1 寻址方式的概念

根据存储程序的概念，计算机在运行程序之前必须把指令和数据（操作数）存放在主存的相应地址单元中。运行程序时，需要根据物理地址从主存中去取指令和数据。如何获得指令和数据的物理地址？

寻址方式

寻找指令和操作数有效地址的方法

寻址方式：寻找指令或操作数有效地址的方法。

寻址技术是计算机设计中硬件对软件最早提供支持的技术之一，**寻址技术包括编址方式和寻址方式。**

寻址方式可分为：

- 指令的寻址
- 操作数的寻址

编址方式

编址方式是指对各种存储设备进行编号的方式。 常用的编址方式：

1. 字编址

   指以存储字为单位对存储器进行编号

2. 字节编址

   不论字长多少位，均以字节为单位对存储器进行编号。

例子

- 例1 对于32K字容量的存储器

（1）若按字节编址，其地址寄存器应是多少位？数据寄存器是多少位？

（2）若按字编址，字长16位。其地址寄存器应是多少位？数据寄存器是多少位？

解：（1） 按字节编址，该存储器的寻址空间为：32K字=32K*8位=2^15*8位

地址寄存器的位数为：15位

数据寄存器的位数为：8位

（2）按字编址，字长16位：该存储器的寻址空间为：2^15*8位= 2^14*16位

地址寄存器的位数为：14位

数据寄存器的位数和字长相等为16位

3.2.3 寻址方式---指令寻址方式

- 计算指令有效地址的方法
- 指令的寻址方式只有两种
  - 顺序寻址
  - 跳跃寻址

指令的顺序寻址

（1）寻址方式：

程序的指令序列在主存顺序存放。

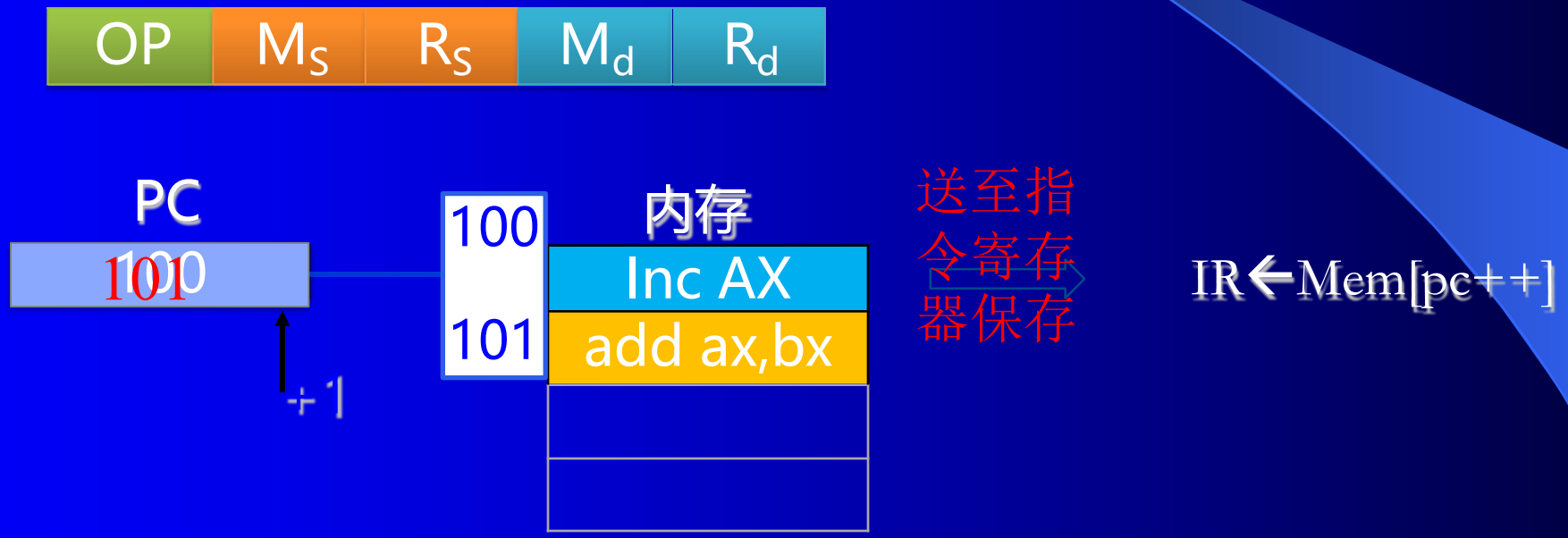
执行时从第一条指令开始，逐条取出并执行。

（2）实现方式

程序计数器（PC）对指令的顺序号进行计数。

PC开始时存放程序的首地址，每执行一条指令，PC 加”1”，指出下条指令的地址，　直到程序结束。

- 2.1 指令的顺序寻址



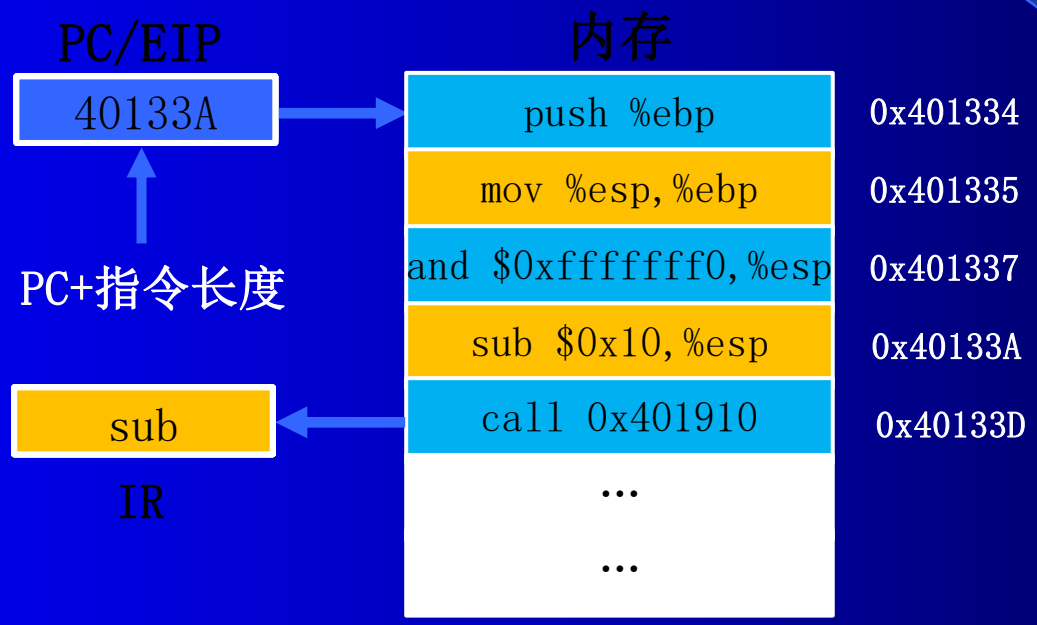
需要深刻理解 “+1” ⇨ 存储1条指令占用的字节单元数

即一条指令的字节长度

比如32位指令字长，PC “+1” =PC+4

– 执行一条指令，PC=PC+1

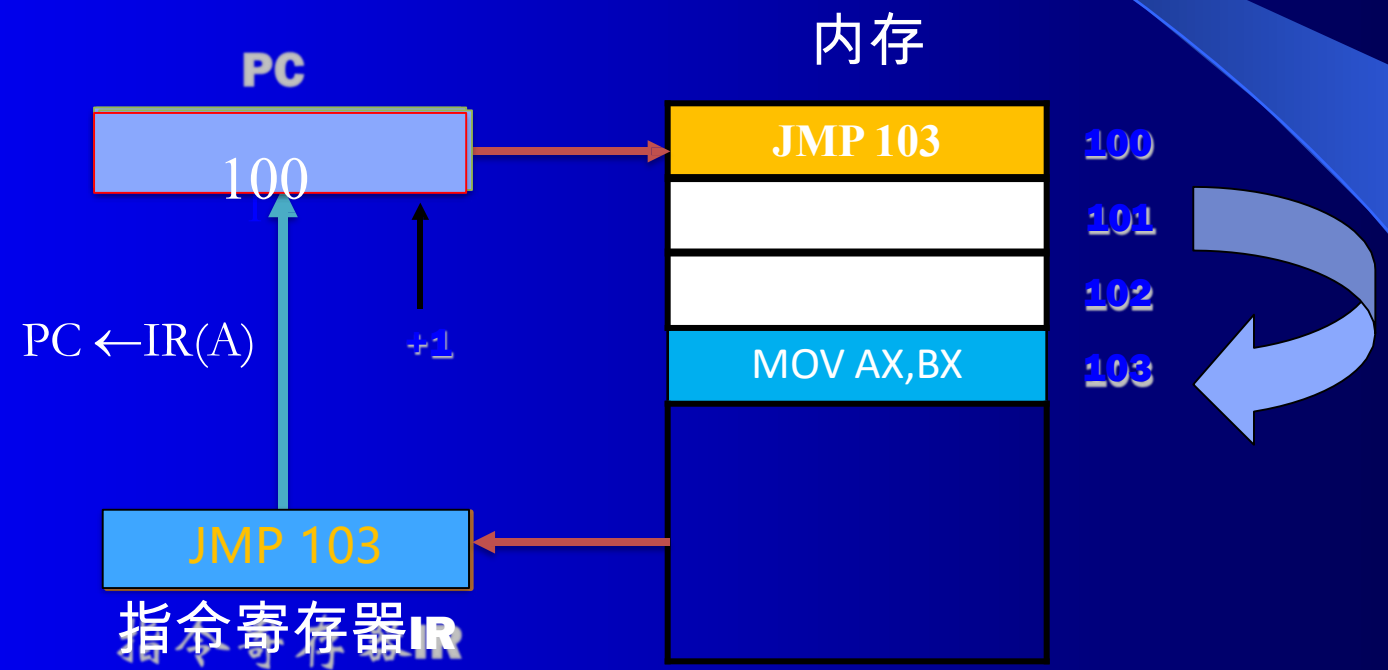
实际上是PC=PC+当前指令字节长度



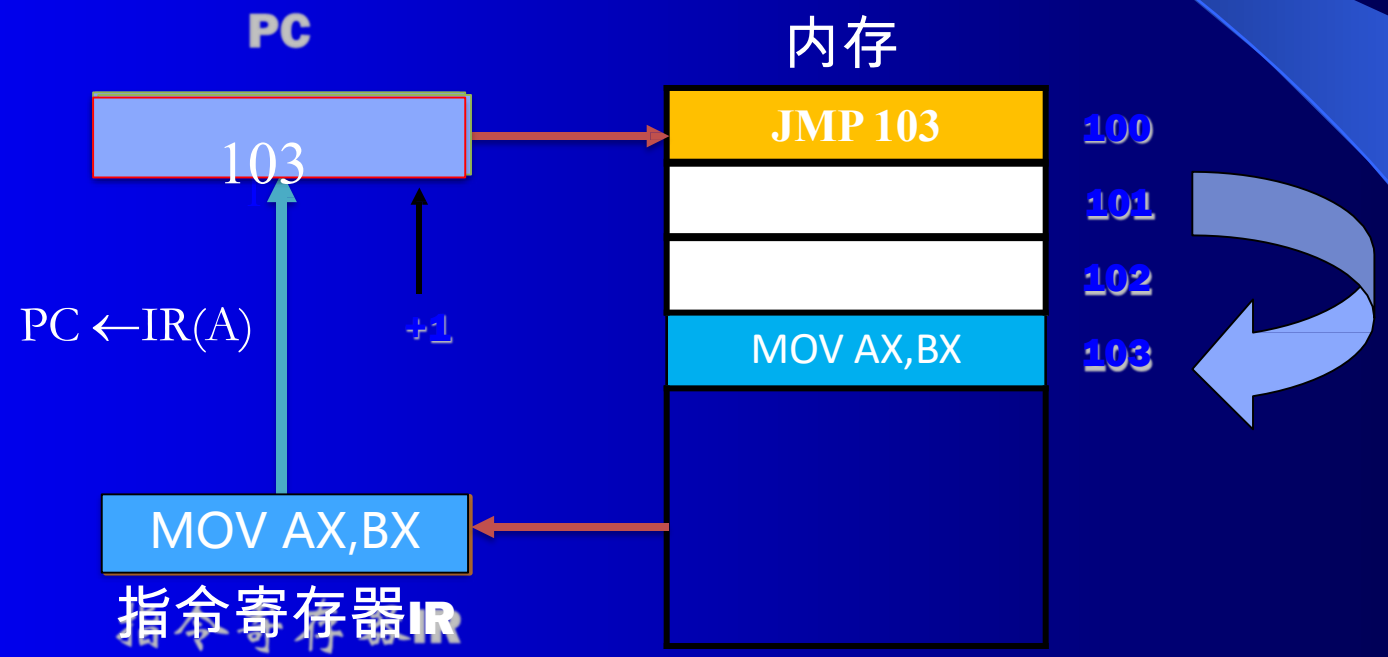
Mem[pc++] → IR

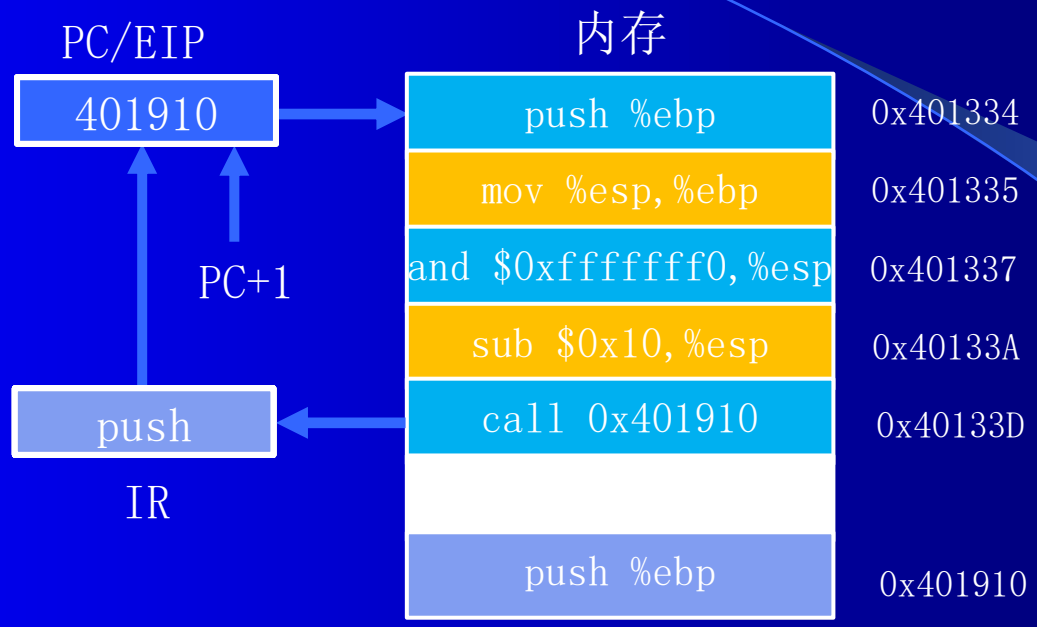
指令的跳跃寻址

当程序中出现分支或循环时，就会改变程序的执行顺序下条指令地址不是PC++得到，而是由指令本身给出跳跃的处理方式是重新修改PC的内容，然后进入取指令阶段。



当程序中出现分支或循环时，就会改变程序的执行顺序下条指令地址不是PC++得到，而是由指令本身给出跳跃的处理方式是重新修改PC的内容，然后进入取指令阶段




PC/EIP
401910
PC+1
push
IR
内存
push %ebp
mov %esp,%ebp
and $0xfffffff0,%esp
sub $0x10,%esp
call 0x401910
push %ebp
0x401334
0x401335
0x401337
0x40133A
0x40133D
0x401910

IR(A) → IR

练习

A

1 直接寻址的无条件转移指令执行的效果是将指令地址送入（　　）

A 程序计数器PC B地址寄存器 C数据缓冲寄存器 D 偏移地址累加器

2、设指令字长为64位，PC顺序寻址时应该增加的常量是(　　)

A. 4　　B. 8　　C. 1　　D. 2

B

练习

B D

3 下列关于指令寻址方式的描述中，正确的是（）多

A 指令的有效地址通过指令中形式地址字段给出

B 程序中有条件和无条件转移采用的就是跳跃式寻址

C指令的不同寻址方式需要通过寻址方式特征为来标识

D顺序结构中CPU依次访问不同指令采用的就是顺序结构

4、直接转移指令JMP，其功能是将指令中的地址部分送入（）

B

A.AC累加器　　B. PC程序计数器

C. MAR地址寄存器　　D. MDR数据缓冲寄存器

3.2.3 指令中的寻址方式----操作数寻址

※指形成操作数地址或寻找操作数的方式；

※1条指令，可能会涉及多种寻址方式；

操作数可能存放在：

（1）包含在指令中，或紧跟着该指令。

（2）在CPU的某个寄存器中。

（3）操作数在主存中，则指令应以某种方式给出主存地址。

（4）操作数在堆栈区中，可以隐含约定由堆栈指针SP提供地址。

（5）操作数在某个I/O接口的寄存器中。

3.2.3 指令中的寻址方式----操作数寻址

操作数寻址过程是指根据地址码字段形成操作数在存储设备中的真实地址。

形式地址：指令地址码字段中的地址称为形式地址（用字母D表示）

有效地址：操作数的真实地址称为有效地址（用字母EA表示）

一般来说,指令中所给出的地址码,并不是操作数的有效地址。因此寻址过程就是把操作数的形式地址变换为操作数的有效地址的过程。

2. 指令中的寻址方式约定

(1) 操作码可隐含说明不同寻址方式

[例] 某机指令操作码的最高两位

- 00：RR型指令，寄存器-寄存器寻址
- 01：RX型指令，寄存器-变址寻址
- 10：SI型指令，基址-立即寻址
- 11：SS型指令，基址-基址寻址

MIPS指令，一般都是采用这种方式。

操作码OP	地址码A /操作数D

（2）指令中可设置寻址方式字段

[例]某机指令的每个地址字段中各设置一个3位的寻址方式说明字段。



或



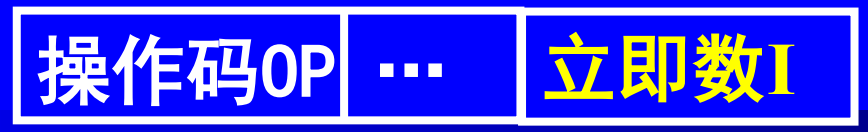
1、操作数常见寻址方式

①立即寻址(Immediate addressing)

指令中直接包含了操作数。助记符I

数在指令中，其长度固定、位数少。

定长格式：



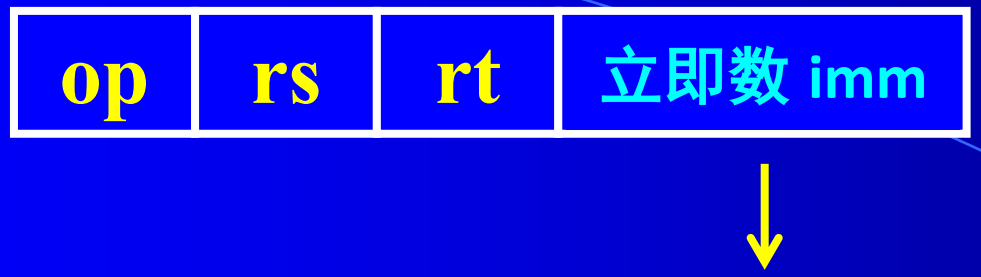
变长格式：



数在基本指令之后，其长度可与指令等长

用来提供偏移量、常数、设置初值等。

- 特点：
  - 取指操作将数据与指令一并读入CPU内部的寄存器，不需要访问主存，指令执行速度快
  - 便于程序设计（一般用于变量赋初值）
  - 数据大小受字段位数限制



addi $rt, $rs, 5

指令功能： $rt ← $rs + imm(符号扩展)

取指令后，直接截取指令中的低16位代码，就能立即得到真值为5的操作数。

因此，得到“5”的方式，就是立即寻址。

②主存直接寻址

指令直接给出操作数在主存中的地址，根据该地址可从存储器中读取或写入操作数。助记符用（A）

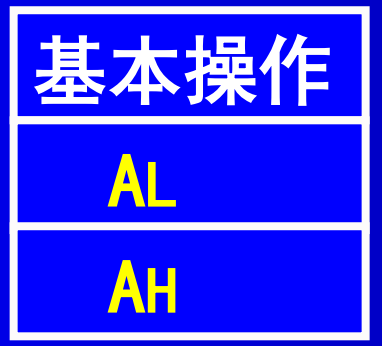
- 寻址过程：操作数地址$\xrightarrow{M}$操作数
- 有效地址EA=A

定长格式



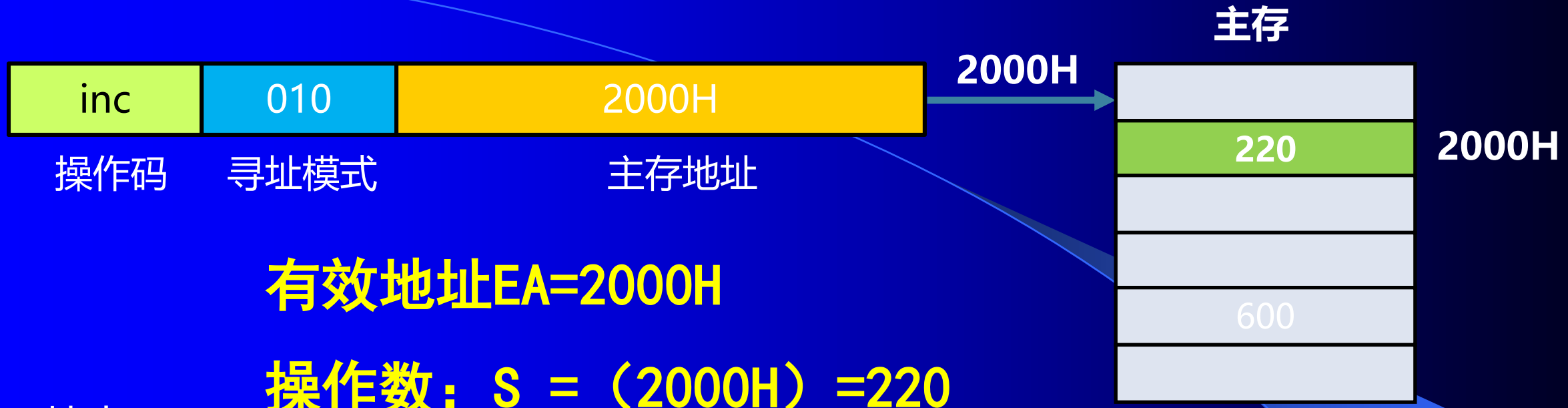
A的位数有限，限制访存范围

变长格式



A的位数可等于指令字长，覆盖整个存储空间

操作数：S =（A）



有效地址EA=2000H

操作数：S =（2000H）=220

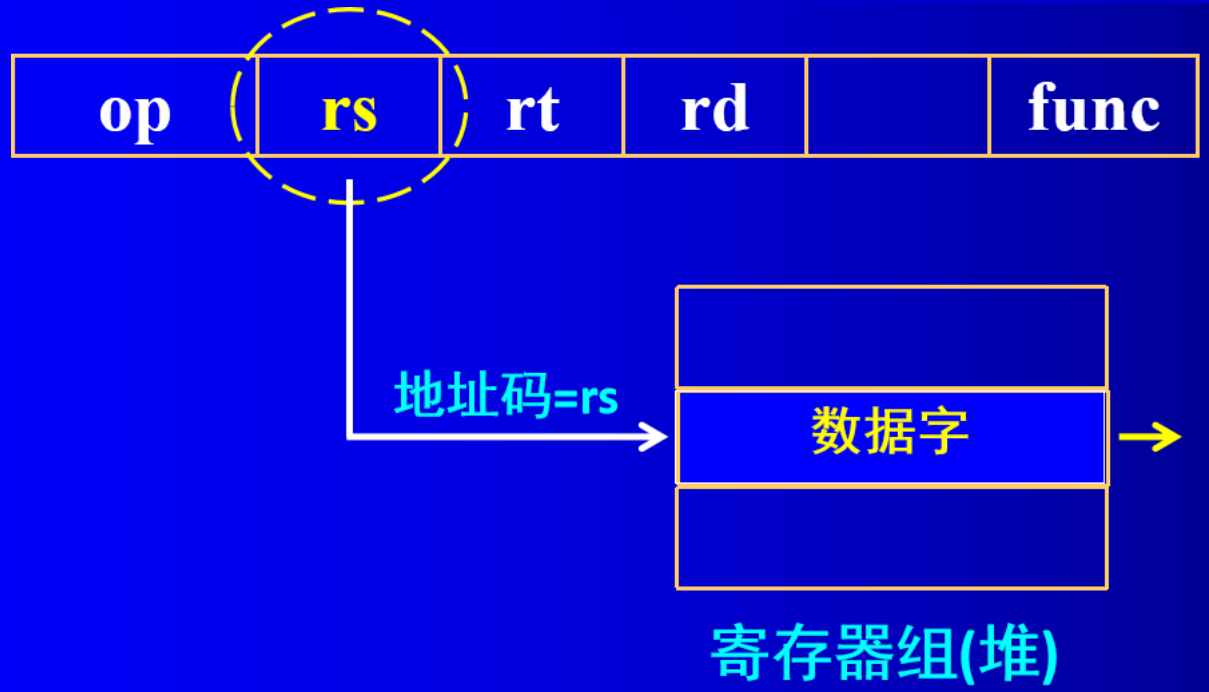
- 特点：
  - 简单直观，便于硬件实现
  - 提供访问主存的操作，访问一次
  - 获得数据要访问主存，指令执行速度慢
  - 地址字段的位数决定了访存空间大小。
  - 数据地址存于指令中，程序和数据在内存里的存放位置受限，灵活性不够

③寄存器直接寻址

指令中给出寄存器号，在指定的寄存器中存放着操作数。助记符用R。

寻址过程：寄存器号----R→操作数

有效地址：EA=R　　R所占位数少；访问R比访问M快





有效地址EA=001

操作数：S =（001）=220

- 特点：
  - 操作数在寄存器中，不需要访问主存，指令执行速度快
  - 所需要的地址码较短，有利于缩短指令字的长度，节省存储空间。

寄存器寻址是计算机中最常用的寻址方式，但由于数量关系，不能为操作数提供大量的存储空间。

④主存间接寻址

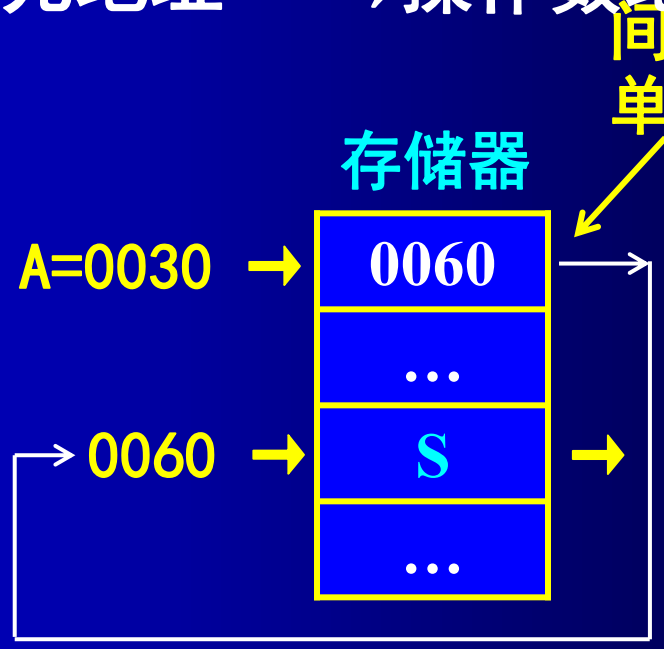
若操作数存放在某个主存单元中，则该主存单元的地址被称为操作数地址。

若操作数地址存放在另一个主存单元中，则该主存单元称为间址单元，间址单元本身的地址称为操作数地址的地址。

指令中给出间址单元地址码（操作数地址的地址），按照该地址访问主存中的某间址单元，从中取出操作数的地址，按照操作数地址再此访问主存，从相应单元中读取或写入操作数。助记符@

有效地址：EA=(A) 寻址过程：间址单元地址----→操作数地址----→操作数

操作码op	间接地址A



需两次访存，速度慢，已淘汰

1次间址

⑤寄存器间址及其变型

由指令给出寄存器号，所指定的寄存器中存放着操作数地址。按照该地址访问主存储器，读取或写入操作数。助记符（R）

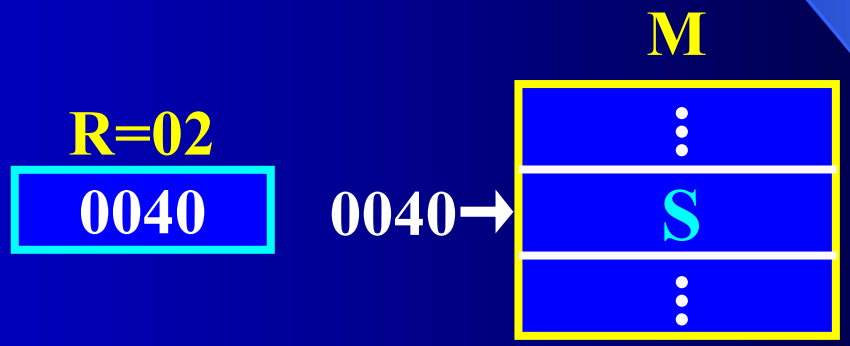


寻址过程：寄存器号 ---R---→ 操作数地址 ---M---→ 操作数

有效地址：EA=（R）

R的地址位数少，R可提供全字长地址码；

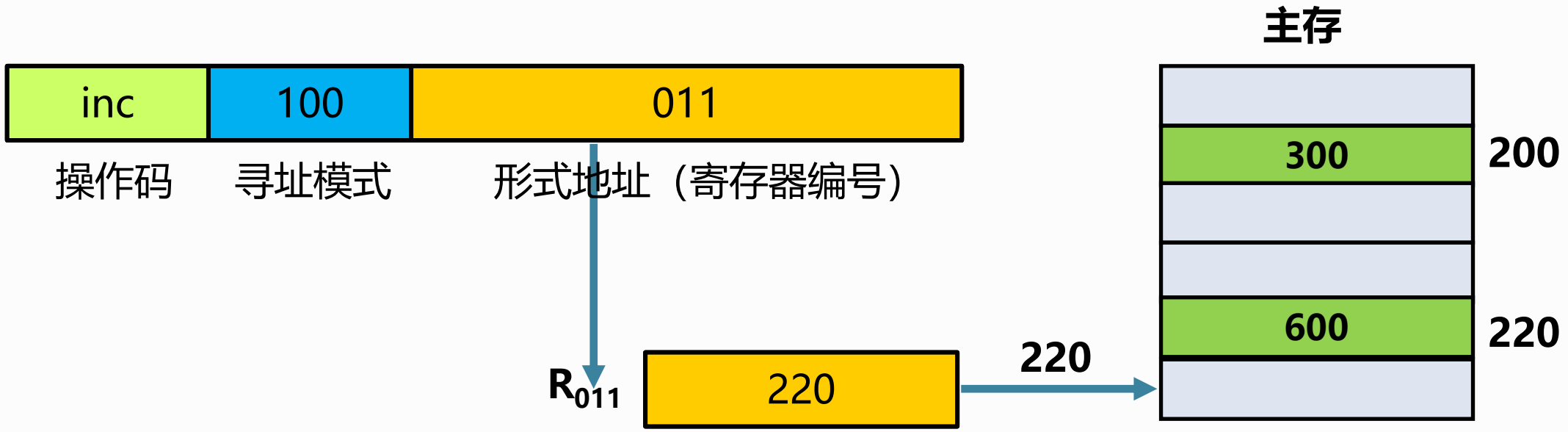
修改R内容比修改M更快。



指针不变(由指令指定)，指针内容可变，使同一指令可指向不同存储单元，以实现程序的循环、共享，并提供转移地址。

寄存器间接寻址特点

- 比主存间接寻址少访问一次主存，由寄存器提供有效地址及修改量，执行速度较快。因此，在编程中，让寄存器充当地址指针；
- 寄存器号位数小于主存地址码，因此CPU目前仍用。



1）自增型寄存器间址

指令中给出寄存器号，被指定的寄存器中存放着操作数有效地址，从寄存器读出操作数地址后，并按照该地址从主存储器中读取操作数，寄存器内容加1 。助记符（R）+。

寻址过程：寄存器号 ---R---→ 操作数地址 ---M---→ 操作数

操作数地址 → 内容加1 → R

有效地址：EA=（R）

（R）+1 →R

2）自减型寄存器间址

指令中给出寄存器号，被指定的寄存器内容减1后作为操作数地址，按照该地址访问存储器，相应的主存单元内容为操作数。助记符-（R）。

寻址过程：寄存器号$\xrightarrow{R}$寄存器内容减1后为操作数地址$\xrightarrow{M}$操作数

有效地址：EA=（R）-1

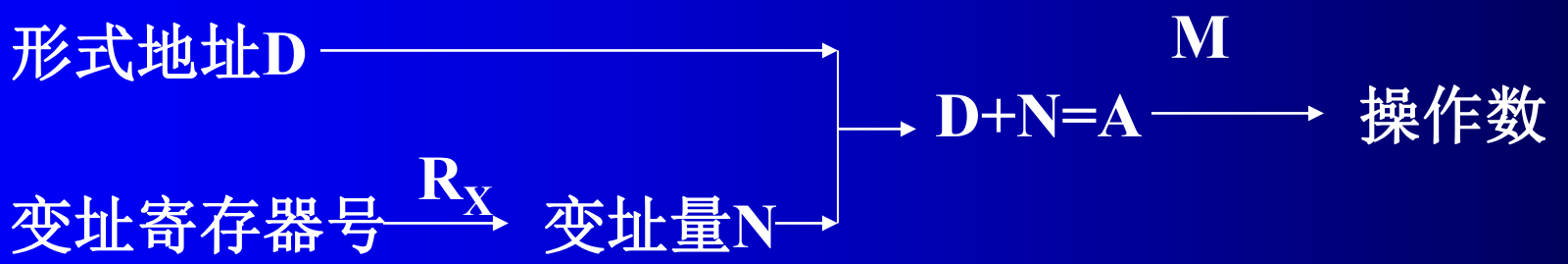
⑥变址寻址---广泛应用的一种寻址方式

指令给出一个寄存器号和一个地址量，寄存器内容与地址量之和为有效地址。助记符X（R）。

格式



变址寄存器号　　形式地址



有效地址： $EA=(R_X)_{变址}+D$

修改量　　基准地址

例. 用变址方式访问一组连续区间内的数组元素。

D为存储区首址；
(Rx)为所访单元距离首址的长度；
Rx初值为0，每访问一个单元，(Rx)+1。



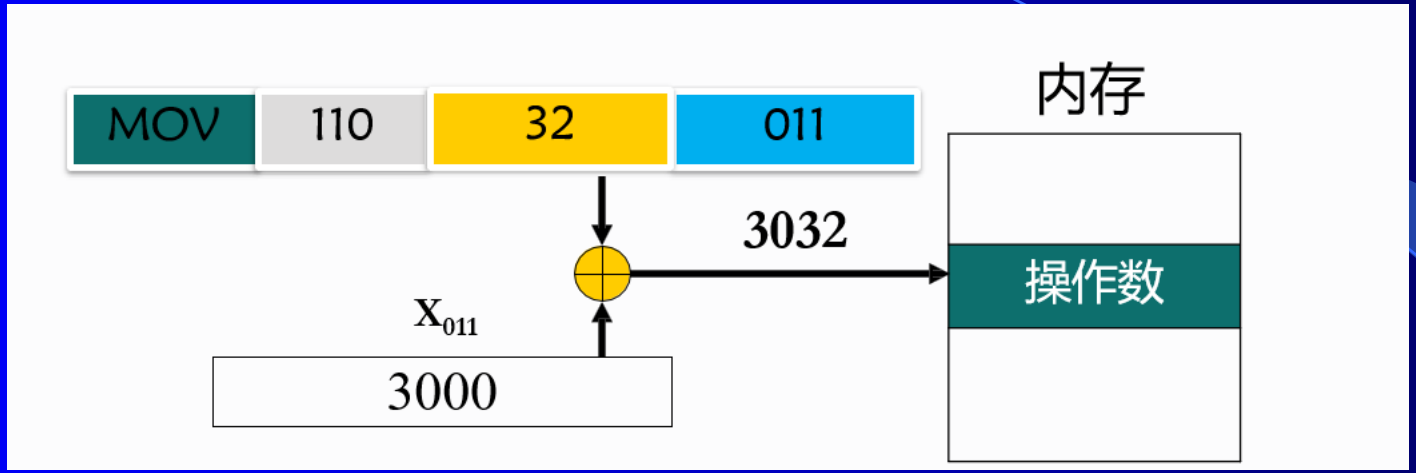
D的位数有限，若不能提供全字长地址码，会使访存空间受到限制。

有效地址：$EA=(R_X)_{变址}+D$

修改量　　基准地址

MOV AX, 32[SI]

有效地址：EA=（RX）变址+D基址



特点：

- 指定一个变址寄存器X用来存放变化的地址，与本指令地址无关，内容可随要求改变
- 变址寄存器提供修改量，指令提供基准，不改变指令即可改变数据的有效地址，可在循环中使用
- 在字符串处理，向量运算等等成批数据处理中非常有用

⑦基址寻址

指令中给出一个变化的形式地址，作为位移量；指令还给出基址寄存器号，基址寄存器内容作为基准地址；基址寄存器内容与形式地址相加，其和为操作数有效地址，按照有效地址访问主存储器，从相应单元中读出操作数或写入数据。助记符A(R)

格式



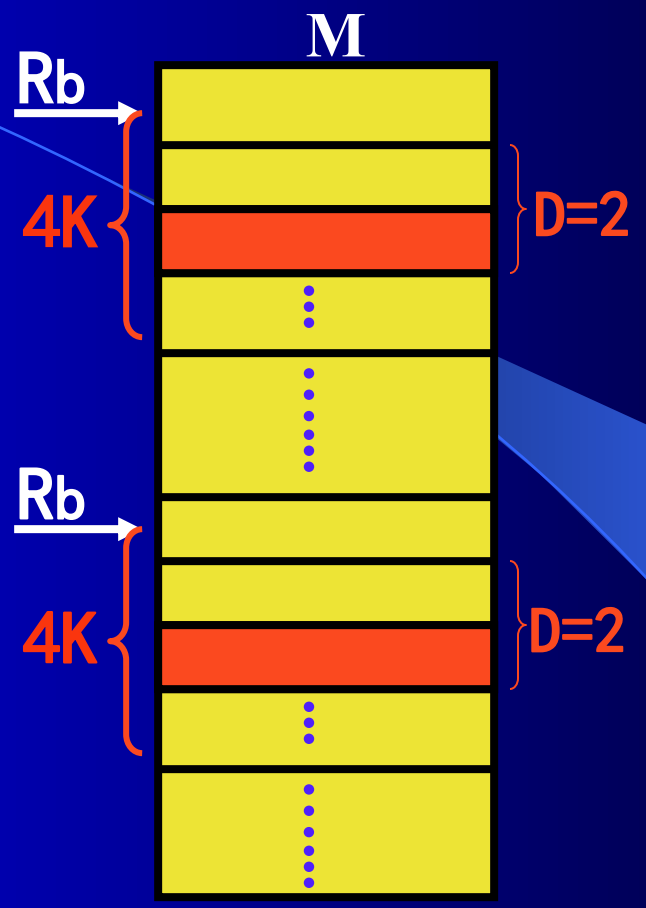
基址寄存器号　位移量

EA =(Rb)+ D

基准地址　相对于基址的位移

EA =(Rb)+ D

基准地址 相对于基址的位移



改变Rb的内容，程序能访问存储空间中任何一个定长区间(4K)。

改变Rb的内容，程序能访问存储空间中任何一个与基址保持相同距离的单元。

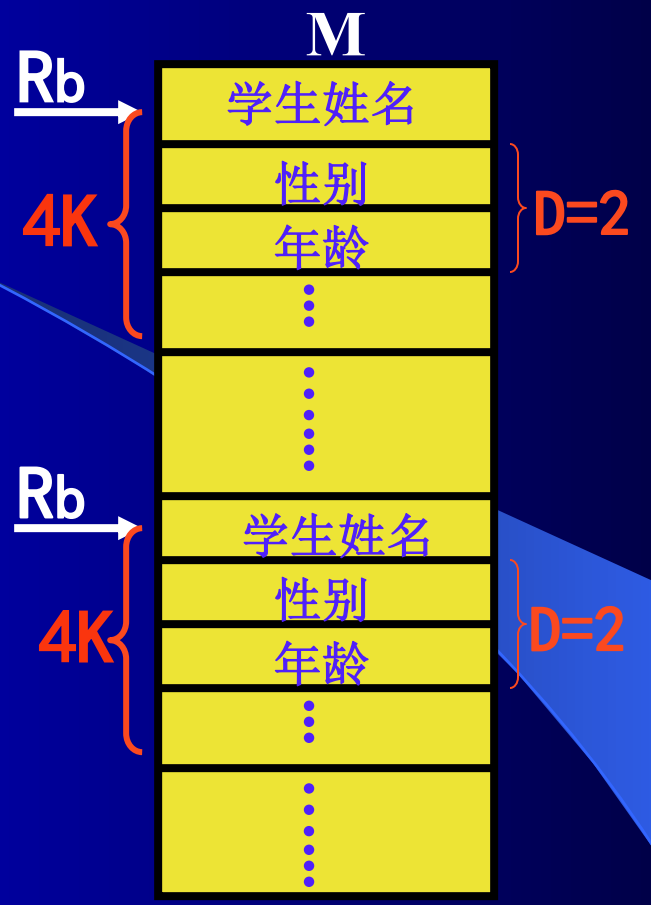
基址寻址：扩展有限字长指令的寻址空间。

便于访问二维数组中某类指定的元素。

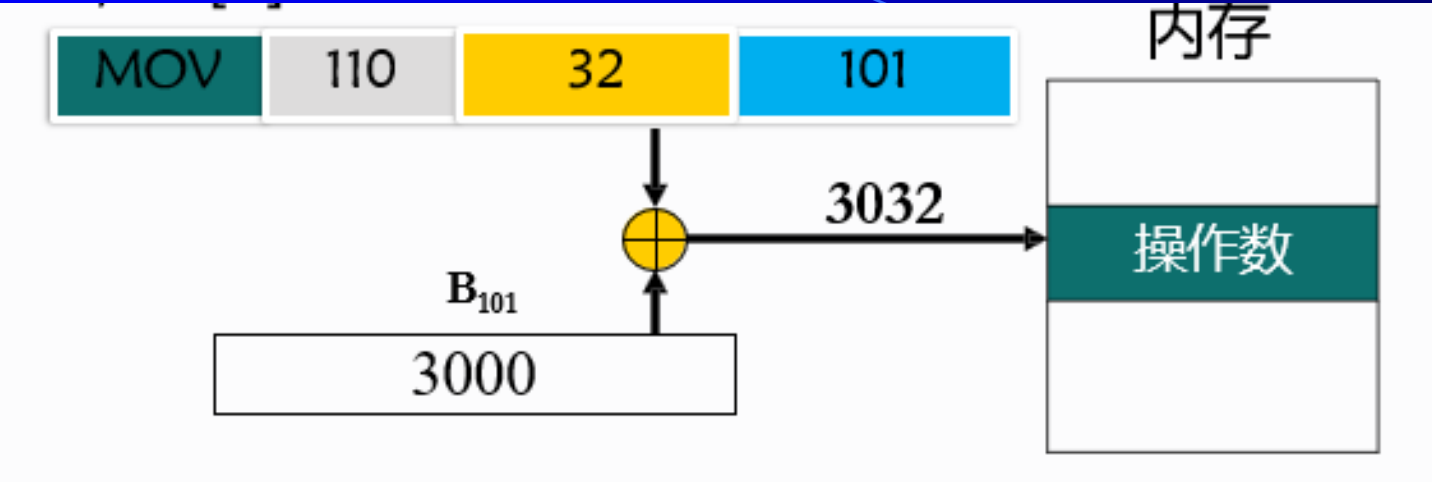
基址方式的一种变化——80X86计算物理地址：

实模式：将段寄存器提供的16位基地址左移4位，与16位偏移相加，得到20位物理地址。

保护模式：将描述符提供的32位基地址与32位偏移相加，得到32位物理地址。



MOV AX, 32[B]



- 特点：

指定一个基址寄存器B用来存放基地址，用指令的形式地址字段D存放一个变化的地址值。

使用基址寄存器可以访问更大的主存空间

对某一程序而言，基址值设定后不变，故要访问不同数据需修改指令中的D

※变址与基址的区别：

有效地址=

寄存器内容(R)+指令中的形式地址D

变址寻址：

指令提供基准量，寄存器提供偏移量；适于处理一维数组。

基址寻址：

指令提供偏移量，寄存器提供基准量；用于扩大有限字长指令的访存空间。

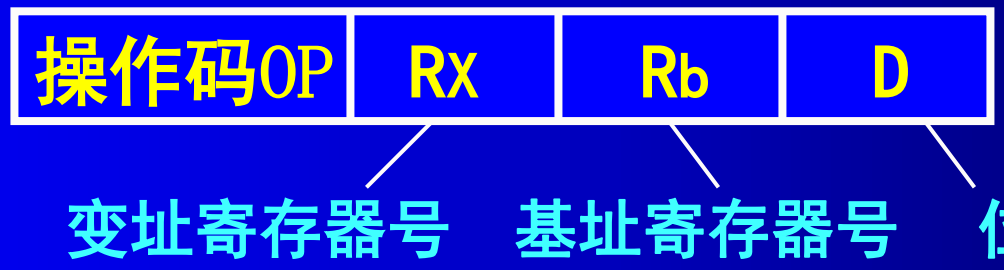
※变址与基址的区别：

基址寻址面向系统的，主要用于程序的重定位以及在有限字长指令中扩大寻址空间。程序运行是该基地址装入基地址寄存器，通过基址寻址方式实现虚拟地址到物理地址的转换。用户通过改变指令中的形式地址D来实现指令或操作数的寻址。

变址寻址是面向用户的，主要用于解决程序循问题，可用于访问字符串、数组、表格等。变址寄存器中的内容由用户设置。程序执行过程中，用户通过改变变址寄存器的内容实现指令或操作数的寻址。

● ⑧ 基址+变址

指令给出两个寄存器号和一个地址量，寄存器内容与地址量之和为有效地址（处理三维数组）。



EA =(Rx)+(Rb)+ D

便于处理三维数组。

例. 访问二维表格。

令：

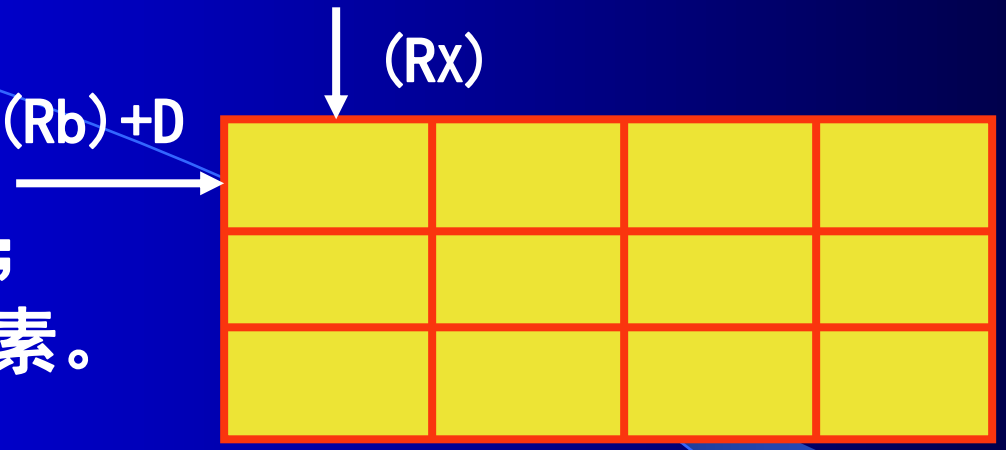
(Rb)+D指向各行起始；

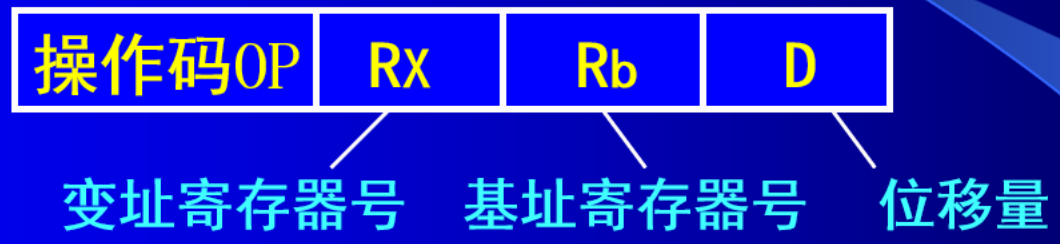
(Rx)指向一行中各元素。

D=0；

Rb初值为表格首址，每访问一行，(Rb)+4；

Rx初值为0，每访问一列，(Rx)+1。



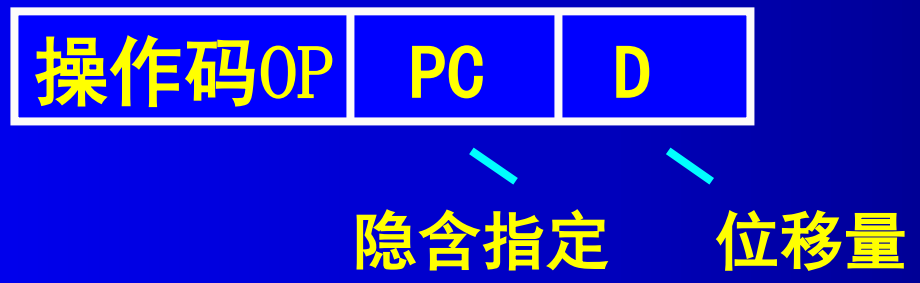


EA =(Rx)+(Rb)+ D

便于处理三维数组。

⑨ PC相对寻址(PC-relative addressing)

指令给出偏移量，PC当前值与偏移量相加得到有效地址。助记符X（PC）



EA =(PC)+D

是一种特殊的基址寻址方式

有效地址相对于PC浮动，编程方便。

⑩页面寻址（伪直接寻址）

指令给出位移量，PC的高位部分与位移量拼接，形成有效地址。

格式

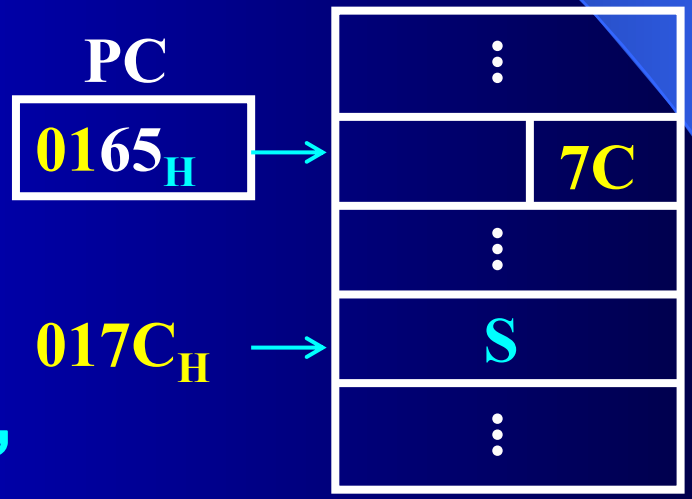


隐含指定　　位移量

$EA = (PC)_H , D$

页号　　页内地址

[例]M为64KB，划分为256页，每页256B，按字节编址。



用于页式存储系统。

寻址速度快，适于组织程序模块，有效利用存储空间。

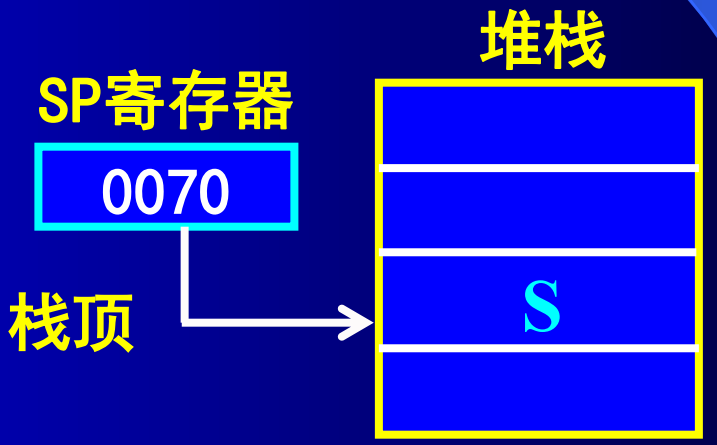
⑪扩展型变址寻址

变址+间址 〈
- 先变址后间址： EA =((R)+D)
- 先间址后变址： EA =(R)+(D)

⑫堆栈间接寻址

操作码 θ	堆栈指针SP

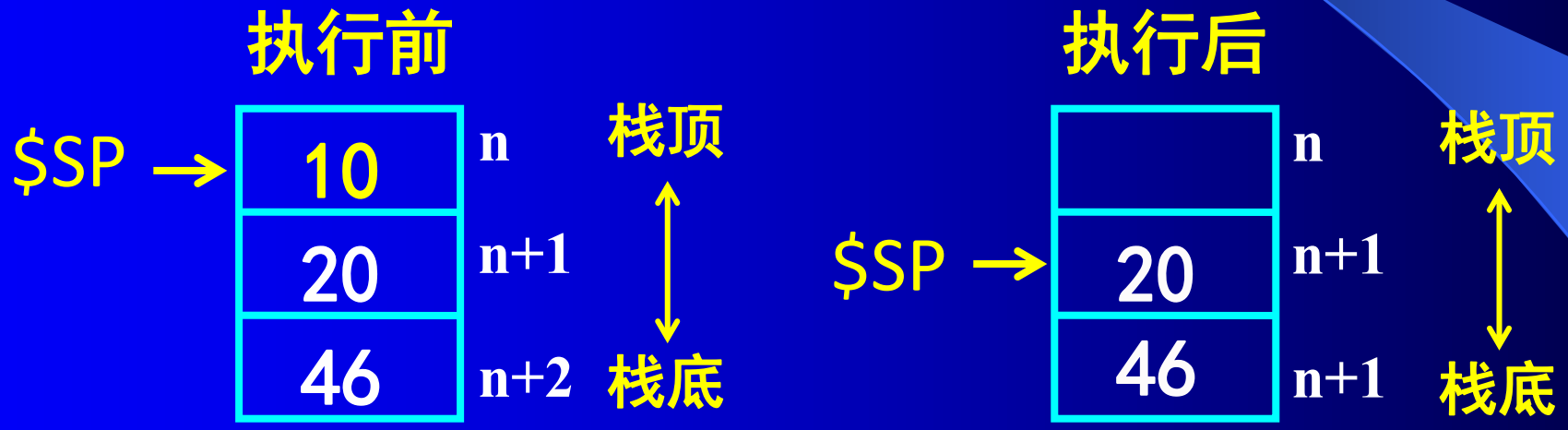
EA =(SP)



［例］ POP (SP)

LW $t0, 0($SP)

SP既可出现在指令中，也可由操作码隐含约定



堆栈有三种方式（向下，向上，栈顶固定）

2. 指令中的寻址方式约定

(1) 操作码可隐含说明不同寻址方式

[例] 某机指令操作码的最高两位

- 00：RR型指令，寄存器-寄存器寻址
- 01：RX型指令，寄存器-变址寻址
- 10：SI型指令，基址-立即寻址
- 11：SS型指令，基址-基址寻址

MIPS指令，一般都是采用这种方式。

（2）指令中可设置寻址方式字段

[例]某机指令的每个地址字段中各设置一个3位的寻址方式说明字段。



练习

- 1 在主存间接寻址中，指令中地址码存放的是操作数。（Y/N）

在主存间接寻址中，指令中地址码存放的是操作数的地址的地址（N）

2、什么是基址寻址和变址寻址？它们的区别是什么？具体面向的应用有何不同？

基址寻址：指令给出一个寄存器号和一个地址量,，寄存器内容与地址量之和为有效地址，（Rb）+D 。变址寻址：指令给出一个寄存器号和一个地址量，寄存器内容与地址量之和为有效地址，（Rx）+D 。

变址与基址的区别：变址是指令提供基准量D（不变），Rx提供修改量（可变）；基址：指令提供位移量D（可变），Rb提供基准量（不变）。

具体应用：变址寻址立足面向用户，基址寻址立足面向系统

练习

• 1、指令系统中支持不同寻址方式的目的主要是( )

D

A.实现存贮程序和程序控制

B.提高指令执行速度

C.提供扩展操作码的可能并降低指令译码难度

D.缩短指令长度，扩大寻址空间，提高编程灵活性

• 2、按字节编制，对单字节指令而言，PC顺序寻址时应该增加的常量是( )

A. 4　　B. 8　　C. 1　　D. 2

C

练习

- 3、操作数在寄存器中的寻址方式称为( )　　C

A.直接 B.间接 C.寄存器 D.寄存器间接

4、下列关于操作数的寻址方式描述正确的是（）　　AC

A 直接寻址方式下，地址字段的位数影响数据的寻址范围

B 间接寻址方式下，地址字段的位数影响数据的寻址范围

C 立即数寻址方式下，地址字段的位数影响数据的寻址范围

D 间接寻址方式下，地址字段的位数不影响数据的寻址范围

寻址方式例题

设某机的指令字长16位，格式、有关寄存器和主存内容如下，X为寻址方式，D为形式地址，请在下表中填入有效地址EA及操作数S的值。



PC=1000

$R_{基}$=2000

100	200
200	500
500	800
1100	100
1102	350
2100	200

寻址方式	X	有效地址EA	操作数
立即	0	S=D	100
直接	1	EA=D=100	200
间接	2	EA=(D)=200	500
相对	3	EA=(PC)+D=1100	100
基址	4	EA=(R)+D=2100	200

练习　若程序计数器PC的值取指阶段修改后为1000：

寻址方式	X	有效地址	操作数
立即	0	S=D	100
直接	1	E=D=100	200
间接	2	E=(D)=200	500
相对	3	E=(PC)+ D =1100	100
变址	4	E=(R)+D= 2100	200

练习　若程序计数器PC的值还需在取指阶段后修改，则为1000+2：

寻址方式	X	有效地址	操作数
立即	0	S=D	100
直接	1	E=D=100	200
间接	2	E=(D)=200	500
相对	3	E=(PC)+D =1102	350
变址	4	E=(R)+D =2100	200

3.2.4 指令的功能和类型

指令的表示

1 二进制位串表示

例如：10010110

2.助记符表示

操作码缩写成助记符（Memonic）来表示，操作数也用符号表示。

如： ADD代表加； SUB代表减；MUL代表乘等

ADD R1,R2

3.2.4 指令的功能和类型

(1) 按指令格式分类

PDP-11：单、双操作数指令等；

(2) 按操作数寻址方式分类

IBM 370：

RR型（寄存器–寄存器）

RX型（寄存器–变址寄存器）

…

(3) 按指令功能分类

传送、访存、I/O、算数逻辑运算、程序控制、处理机控制等指令。

●（3.1）数据传送类指令

源地址对应的存储单元 —数 据→ 目的地址对应的存储单元

主要包括:

取数指令、存数指令、数据传送（单字、成组）、数据交换和堆栈操作等。

主要用来实现:

寄存器之间、存储器单元之间以及寄存器-存储器单元的数据传送。

设计传送指令时需要考虑：

（1）规定传送范围

（2）指明传送单位

（3）设置寻址方式

MIPS32中

只有lw和sw这两个指令可以访问存储器

● (3.2)输入/输出(I/O)指令

主机 

外部设备

设计时需考虑：

(1) I/O指令对设备的适应性

如何用通用I/O指令实现对各种具体设备的控制？

✓I/O指令中留有扩展余地

指令中某些字段事先不定义，需要时再约定其含义。

用于外设种类、数量不多的场合。

✓把设备抽象化、透明化处理（接口中设置控制/状态/数据寄存器）

（2）主机对外设的寻址方式

寻找I/O接口中的寄存器的方式。

↓

I/O端口

该类寄存器的编号，也称为I/O端口的地址（简称端口地址）

如何为I/O端口分配地址？

- ✓单独编址
- ✓统一编址

（3）I/O指令的设计思路

思路1：设计专用的I/O指令　　（显式I/O指令）

针对端口自行单独编址，用I/O指令访问I/O端口。
指令中说明I/O类型，并给出端口地址。

思路2：用传送指令替代I/O指令（隐式I/O指令）

针对端口和主存统一编址，用访存指令访问I/O端口，
故不必再设计专用的I/O指令。

思路3：通过IOP进行I/O操作控制

需设计两级I/O指令
- CPU控制IOP(如启动、停止等)
- IOP控制具体I/O操作(如保存等)

● (3.3)算术\逻辑运算指令

（1）算术运算指令

设计时需考虑操作数类型、符号、进制等，运算结束后设置CPU相应状态标志寄存器。

如基本的加法(add, addi)，减法(sub)等指令

（2）逻辑运算指令

如与(and)，或(or)，异或(xor)等指令

常用来对码位的设置和条件判断等操作。

● (3.4) 程序控制类指令

主要作用： 控制指令的执行流程。

（1）转移指令

- 无条件转移 ： 操作码+转移地址
- 条件转移 ： 操作码+转移地址+转移条件
- 循环 ： 转移条件为循环计数值

（2）转子指令与返回指令

转子：即调用，操作码+子程序入口

返回：操作码+返回地址（堆栈的顶单元中）

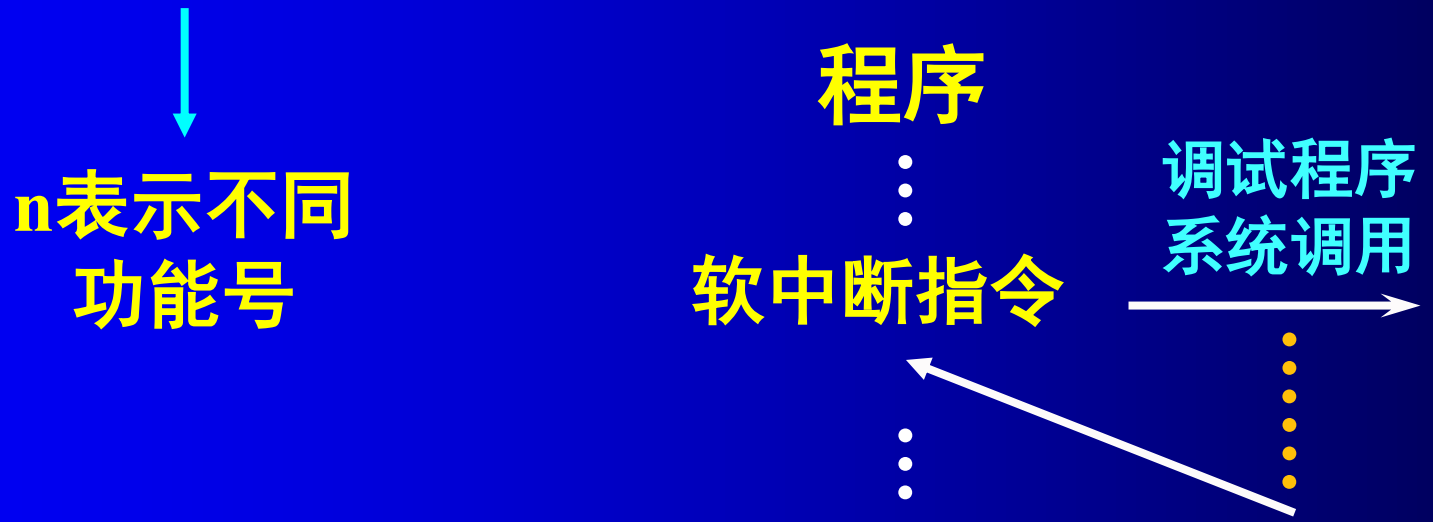
同一条返回指令应能提供多个不同的返回地址（条件返回），*一般用堆栈存放返回地址。*

（3）软中断指令

早期主要用于程序的调试。

现在常常用于系统功能调用。

常以INT n的指令形式出现在程序中。



- (3.5) 控制处理机的专用指令

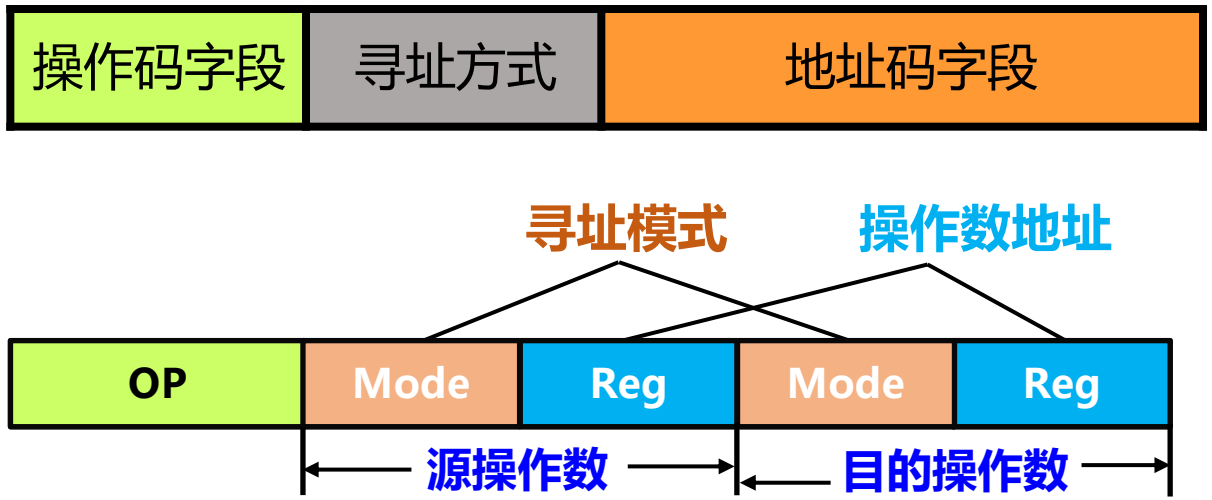
如CPU状态字标志位的清楚、修改，空操作指令NOP、暂停HLT、等待WAIT、总线锁定LOCK等。

- (3.6) 面向操作系统的指令

提供给操作系统专用，如访问系统寄存器、检查保护属性、存储管理等。

扩展知识 指令格式设计

1 指令格式设计的主要内容



- 根据指令数量的要求及是否支持操作码扩展，确定操作码字段的位数
- 根据对操作数的要求确定地址码字段的个数
- 根据寻址方式的要求，为每个地址码字段确定寻址方式字段位数
- 确定采用定长指令还是变长指令

1 指令格式设计

2 指令格式设计举例

例1 某机字长32位，采用三地址指令，支持8种寻址操作，完成60种操作，各 寻址方式均可在2K主存范围内取得操作数，并可在1K范围内保存运算结果。 问应采用什么样的指令格式？指令字长最少应为多少位？执行一条指令最多要 访问多少次主存？

1 指令格式设计

2 指令格式设计举例

例1 某机字长32位，采用三地址指令，支持8种寻址操作，完成60种操作，各 寻址方式均可在2K主存范围内取得操作数，并可在1K范围内保存运算结果。 问应采用什么样的指令格式？指令字长最少应为多少位？执行一条指令最多要 访问多少次主存？

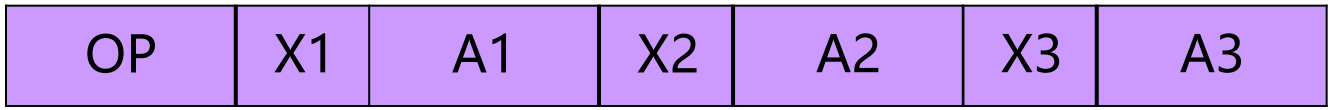
解：根据题目条件，指令格式如下：

OP	X1	A1	X2	A2	X3	A3

2 指令格式设计举例

例1 某机字长32位，采用三地址指令，支持8种寻址操作，完成60种操作，各 寻址方式均可在2K主存范围内取得操作数，并可在1K范围内保存运算结果。 问应采用什么样的指令格式？指令字长最少应为多少位？执行一条指令最多要 访问多少次主存?

解：根据题目条件，指令格式如下：



OP = 6位

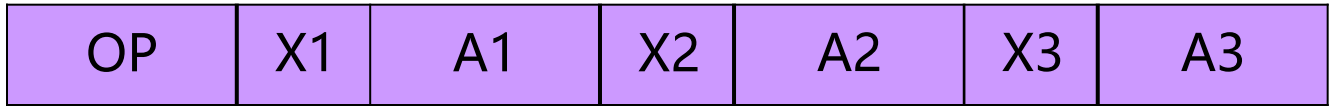
X1 = X2 = X3 =3 ,共 9位

A1=A2=11, A3=10位，共 32位

⟹ 47位

1 指令格式设计

2 指令格式设计举例

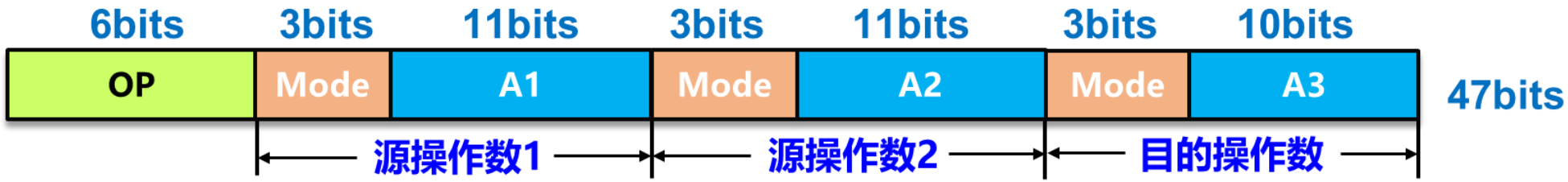


OP = 6位

X1 = X2 = X3 =3 ,共 9位

A1=A2=11, A3=10位，共 32位

⟹ 47位



47位指令占两个存储字，取指访存2次。

取2个源操作数访存2次，写结果访存1次，共5次

2 指令格式设计举例

例2 字长16位，主存64K，指令单字长单地址，80条指令。寻址方式有直接、 间接、相对、变址。请设计指令格式

2 指令格式设计举例

例2 字长16位，主存64K，指令单字长单地址，80条指令。寻址方式有直接、 间接、相对、变址。请设计指令格式

解：80条指令 ⇒ OP字段需要7位($2^7=128$) 4种寻址方式 ⇒ 寻址方式特征位2位

单地址位长度:16-7-2 =7位

1 指令格式设计

2 指令格式设计举例

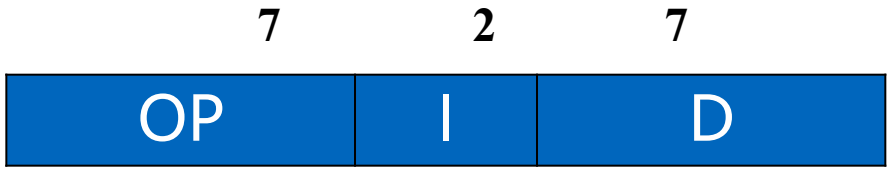
例2 字长16位，主存64K，指令单字长单地址，80条指令。寻址方式有直接、 间接、相对、变址。请设计指令格式

解：80条指令 ⇒ OP字段需要7位($2^7=128$) 4种寻址方式 ⇒ 寻址方式特征位2位

单地址位长度:16-7-2 =7位

指令格式：

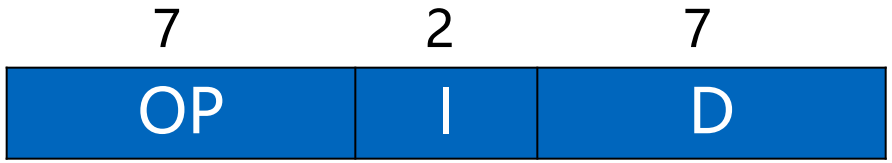




1 指令格式设计

② 指令格式设计举例

例2 字长16位，主存64K，指令单字长单地址，80条指令。寻址方式有直接、间接、相对、变址。请设计指令格式



设PC寄存器16位　　变址寄存器16位

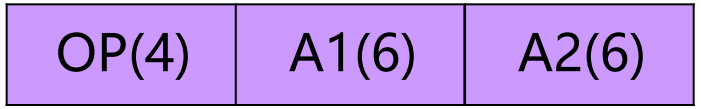
- 相对寻址　EA=（PC）+A，寻址范围为：64K
- 变址寻址　EA=（R）+A，　寻址范围为：64K
- 直接寻址　EA = A，寻址范围为　　：128
- 间接寻址　EA = （A），　寻址范围为　　：64K

设计评价：

- 访问主存的方式太多
- 缺立即数寻址

2 指令格式设计应用举例

例3 设某指令系统指令字长16位，每个地址码为6位。若要求设计二地址指令15条、一地址指令34条，问最多还可设计多少条零地址指令?



② 指令格式设计应用举例

例3 设某指令系统指令字长16位，每个地址码为6位。若要求设计二地址指令15条、一地址指令34条，问最多还可设计多少条零地址指令?

OP(4)	A1(6)	A2(6)

解: 操作码按从短码到长码进行扩展编码

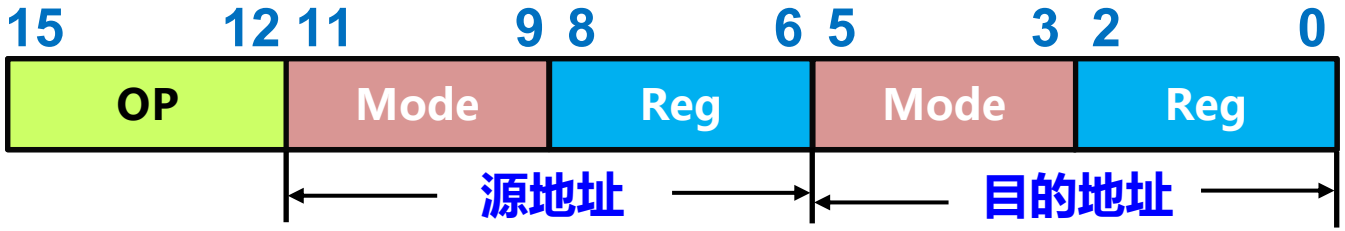
假定二地址指令编码: (0000 – 1110)　　共15条，1111作为扩展标识;

则一地址指令编码的全集可表示为: 1111 000000 – 111111(假定扩向A1);　因一地址指令只需34条，可将全集中多余的30种编码作为向零地址指令扩展的标识，假定为: 100010 – 111111

故最多可支持的故零地址指令数为:　30×2^6 条

2 指令格式设计应用举例

例4 分析以下指令格式及寻址方式特点?



1）二地址指令;
2）操作码可指定16条指令;
3）源和目的均有8种寻址方式;
4）源地址寄存器和目的地址寄存器均有8个;
5）可寻址范围为1~64K